A Tree-Based Number System over the Combinators K and S, and Primality as Unreachability

Working paper v0.5, Project secret1, August 11, 2026. Appendix C synchronized against the shared proof module at 3,337 lines.

Abstract

We develop a number system built on binary trees over the combinators K and S, in which a natural number is an equivalence class of trees, and in which the choice of the equality relation is the central design question. The work proceeds in two stages. First, in a warm-up world of bare binary trees, we define multiplication as uniform grafting and prove that n is prime exactly when no binary tree with n leaves admits a grafting decomposition; the equality that turns trees into numbers there is associativity. Second, we transfer the question to combinatory logic: with iterator numerals $\bar{n} = (SB)^n(KI)$ and multiplication given by the composition combinator B, weak reduction turns out to be too fine an equality (the class of 6 splinters into distinct normal forms), while full extensional equality is undecidable. We close the gap with a system R_arith of four local rewrite rules, machine-verified to be pointwise valid, locally confluent by a critical-pair analysis, and terminating on the tested fragment. The induced equality =wR is decidable there, carries all of arithmetic, and makes commutativity and associativity emergent rather than postulated. Finally we invert the system: primality of n becomes the unreachability, from the numeral $\bar{n}$, of any product-shaped term $S(K\,\bar{m})\bar{k}$ with m, k ≥ 2, and confluence upgrades this from an empirical non-observation over an infinite search space to a provable statement. Exhaustive bounded searches (99,012 terms for n = 5) and step-verified traces expose the 'dead-end anatomy' of primes: every stride k strands on a residual comb of length n mod k. Since version 0.3, the core schematic results, Theorem A, the adequacy of the numerals, and the universal validity of all four rules, are additionally compiler-checked theorems in a total Idris2 proof module (Section 7, Appendix C); version 0.4 adds the fragment form of the unreachability theorem, proved through a value invariant with no appeal to confluence. We conclude with an assessment of how far this takes us, and of what the unary encoding still hides.

1. Introduction

The goal of this project is a number system that lives on binary trees built from the combinators K and S, and that makes it possible to give a structural account of the prime numbers. From the outset, one concept was singled out as central: the equality of trees with respect to their role as number representations. This paper records the first complete arc of the project, from design decisions through machine-checked experiments to two precise structural characterizations of primality.

Three design questions had to be settled before anything could be computed.

First, what is a number? We chose the equivalence-class reading: a number is not a single tree but a class of trees identified by a suitable equality. This choice places the equality relation at the heart of the system, which matches the project's guiding intuition.

Second, what is multiplication, structurally? A notion of 'prime' only makes sense relative to a composition operation. We study two candidates that turn out to be two faces of the same idea: graft-

ing of bare trees (substitute one tree into every leaf of another), and functional composition of combinatory numerals (the combinator B).

Third, and decisively: which equality decides decomposability? The number 6 equals 2 times 3 only modulo some identification of trees; the tree that canonically represents 6 does not visibly contain a 2 and a 3. Whether a factorization 'exists' therefore depends entirely on which representatives the equality admits into the class. The main technical contribution of this paper is a concrete, decidable equality, called =wR, that is exactly strong enough to carry arithmetic, and the reformulation of primality as a reachability property of the associated rewriting system.

All computational claims below were machine-checked with a small rewriting engine; Appendix A describes the verification discipline. Since version 0.3, the paper's core theorems are additionally machine-proved: Theorem A, the adequacy of the numerals, and the pointwise validity of all four rules of R_arith are compiler-checked theorems in a total Idris2 module, joined in version 0.4 by the fragment form of Theorem B, reported in Section 7 and reproduced verbatim in Appendix C.

2. Preliminaries

2.1 Binary trees

A binary tree over a single leaf symbol • is either the leaf itself or an ordered pair (s t) of two binary trees. Trees with n leaves are counted by the Catalan numbers: there are Catalan(n−1) of them, so 1, 1, 2, 5, 14, 42, 132, … for n = 1, 2, 3, 4, 5, 6, 7, and so on. Binary trees form the free magma on one generator: the set of all ways to parenthesize a sequence, with pairing as the only operation and no laws imposed.

2.2 Combinatory terms and weak reduction

Combinatory logic (CL) uses the same tree shape with two leaf symbols. A CL term is either the constant K, the constant S, a variable, or an application (t u) of one term to another. We write applications left-associatively, so `S K K` abbreviates `((S K) K)`. Every CL term is literally a binary tree whose internal nodes are applications and whose leaves are drawn from {K, S, variables}. Standard references for combinatory logic and weak reduction are Curry and Feys [1], Hindley and Seldin [2], and Bimbó [14].

Computation is by weak reduction, generated by two contraction rules applied anywhere inside a term:

```
K a b     ->  a
S a b c   ->  (a c)(b c)
```

A term with no contractible subterm is a normal form. Weak reduction is confluent (the Church-Rosser property): if a term reduces to two different terms, those reduce further to a common term. Consequently a term has at most one normal form, and two terms are weakly convertible (written =w) exactly when they share a reduct; for normalizing terms, exactly when their normal forms coincide syntactically.

Two derived combinators recur throughout. The identity is I = SKK; indeed I x = SKK x ⟶ (Kx)(Kx) ⟶ x. The composer is B = S(KS)K. Its behavior is worth tracing once in full, because the trace explains a phenomenon that drives Section 4:

```
B x      =  S(KS)K x   ->  (KS x)(K x)  ->  S(Kx)
B x y    =  S(Kx) y                          (stuck: S has only two arguments)
B x y z  =  S(Kx) y z  ->  (Kx z)(y z)  ->  x(y z)
```

So B x y z computes the composition x(y z), but the two-argument stage B x y freezes at the normal form S(Kx)y, waiting for a third argument.

2.3 Three notions of equality

Three equalities on CL terms will be graded against each other.

Syntactic identity: two terms are equal only if they are the same tree. This is trivially decidable and far too fine for arithmetic.

Weak convertibility =w: the equivalence generated by weak reduction, characterized above via normal forms. Decidable on normalizing terms.

Extensional equality =ext: two terms are equal if they behave identically as functions, that is, applying both to the same arguments always yields weakly convertible results. This is the equality under which the numerals of Section 4 obey ordinary arithmetic, but it is undecidable in general.

The project's central question can now be phrased sharply: where between =w and =ext does the arithmetic of tree-numbers actually live, and can that place be described by local tree-rewriting rules?

3. World One: numbers from bare trees

Before involving K and S, we test the intuitions on bare binary trees.

3.1 Grafting as multiplication

For trees s and t over the single leaf •, define the graft s ⊗ t as the tree obtained by replacing every leaf of s with a copy of t. Two facts are immediate. The leaf counts multiply: leaves(s ⊗ t) = leaves(s) · leaves(t), because each of the leaves(s) positions receives leaves(t) leaves. And grafting is associative: (s ⊗ t) ⊗ u = s ⊗ (t ⊗ u), since both sides replace every leaf of s by t-with-its-leaves-replaced-by-u. (Both facts were additionally machine-checked, the second on 200 random triples.)

Grafting is not commutative at the level of trees. Writing trees in bracket notation, the two trees with three leaves are ((••)•) and (•(••)), and one computes four pairwise distinct grafts of total leaf count 6:

```
2 (x) 3 :   ((.(..))(.(..)))     and     (((..).)((..).))
3 (x) 2 :   ((..)((..)(..)))     and     (((..)(..))(..))
```

The results agree in leaf count but not in shape: 2 ⊗ 3 duplicates a 3-leaf pattern under a fresh root, while 3 ⊗ 2 replaces the leaves of a 3-leaf pattern by cherries.

3.2 Indecomposable trees exist at every composite size

Call a tree decomposable if it equals s ⊗ t with leaves(s) ≥ 2 and leaves(t) ≥ 2. Not every tree of composite leaf count is decomposable. The left comb with six leaves, (((((••)•)•)•)•), is not: in any proper graft s ⊗ t, both children of the root are copies of subtrees of t grafted below s's root children, so each root child has at least leaves(t) ≥ 2 leaves; a comb, however, has a bare leaf as one root child. The same argument shows that any tree with a leaf attached directly to the root is indecomposable.

3.3 The census, and the first characterization of primality

An exhaustive computation over all Catalan(n–1) trees, for n from 2 to 12, counts the decomposable ones:

n	decomposable	total trees
2	0	1
3	0	2
4	1	5
5	0	14
6	4	42

7	0	132
8	9	429
9	4	1,430
10	28	4,862
11	0	16,796
12	98	58,786

The zeros sit exactly at the primes, and this is a theorem rather than a coincidence.

Theorem A. For n ≥ 2: n is prime if and only if no binary tree with n leaves is decomposable.

Proof. If some tree with n leaves equals s ⊗ t nontrivially, then n = leaves(s) · leaves(t) is a nontrivial factorization, so n is composite. Conversely, if n = a · b with a, b ≥ 2, pick any tree s with a leaves and any tree t with b leaves; then s ⊗ t is a decomposable tree with n leaves. ∎

Two readings of Theorem A shaped everything that follows. First, primality here is a universal statement about an entire Catalan family: not one tree fails to decompose, but all of them do. A factorization of n is a 'uniform cut line' inside some tree with n leaves, a level at which the same subpattern hangs from every position; n is prime when no member of the family possesses such a cut. Second, the census shows that decomposable trees are rare and their density tends to zero: at the level of raw trees, almost everything is 'prime'. Ordinary arithmetic, with its commutativity and its familiar distribution of primes, only appears after trees are identified by a suitable equality.

3.4 The equality that creates numbers here is associativity

Which equality? Impose the single local rewrite (x y) z x (y z), that is, associativity. Its equivalence classes are exactly the leaf-count classes: associativity rewrites any tree into the right comb with the same number of leaves, and leaf count is invariant under the rule. The quotient of the free magma by associativity is the free semigroup on one generator, isomorphic to the positive integers, and grafting descends to ordinary multiplication on the quotient. In this world, then, a number is an association class of trees, and n is composite exactly when its class contains at least one decomposable representative. One local rule creates the numbers; this is the pattern we will look for again inside combinatory logic.

4. World Two: combinatory numerals and the splintering of 6

4.1 Numerals

Inside CL we use the iterator numerals

```
0̄ = KI          succ = SB          n̄ = (SB)ⁿ(KI)
```

so that 1̄ = SB(KI), 2̄ = SB(SB(KI)), and so on. Each n̄ is already a weak normal form: every S in it has at most two arguments, every K at most one. The behavioral specification is n̄ f x =w f^n x, verified as follows: 0̄ f x = KI f x ⟶ I x ⟶ x, and (SB n̄) f ⟶ B f (n̄ f), so (SB n̄) f x ⟶ f (n̄ f x); by induction, n̄ iterates f exactly n times.

Two structural observations connect back to World One. The numeral n̄ is literally the right comb over the link SB with terminator KI: the associativity normal form of Section 3 reappears as the canonical shape of numbers. And addition of canonical numerals is literal concatenation: replacing the terminator KI inside ā by the comb b̄ yields, as a syntactic tree, exactly the comb of a + b. At the level of canonical representatives, addition needs no computation at all.

4.2 Multiplication and the frozen product

Composition multiplies iterators: $(\bar{m} \circ \bar{n})\, f = \bar{m}\,(\bar{n}\, f)$ applies f exactly m · n times. So we define the product operation on terms by $\otimes := B$, that is, $m \otimes n$ is the term $B\,\bar{m}\,\bar{n}$.

Now reduce $B\,\bar{2}\,\bar{3}$ by the trace of Section 2.2:

```
B 2̄3̄ =  S(KS)K 2̄3̄ ->  (KS 2̄)(K 2̄) 3̄ ->  S(K 2̄) 3̄       (normal form)
```

The reduction freezes at $S(K\,\bar{2})\,\bar{3}$, because S with two arguments is stuck. This normal form is a different tree from $\bar{6} = (SB)^6(KI)$. Both are normal forms; by confluence, distinct normal forms are never weakly convertible. Therefore:

Observation. $2 \otimes 3$ and 6 are not weakly equal. Likewise $B\,\bar{3}\,\bar{2}$ normalizes to $S(K\,\bar{3})\,\bar{2}$, a third distinct normal form. Yet all three terms, applied to fresh variables f and x, weakly reduce to the same term f(f(f(f(f x))))). (All statements machine-checked.)

The consequences are the pivot of the paper. Weak convertibility is too fine: the intended 'class of six' splinters into infinitely many =w-islands, among them $\bar{6}$, $S(K\,\bar{2})\bar{3}$, and $S(K\,\bar{3})\bar{2}$. The non-commutativity of World One survives literally: the two factor orders freeze into different trees. But the freeze is also a feature: a product-shaped representative keeps its factors visible as intact subtrees, which is precisely the kind of representative a structural theory of factorization needs. What is missing is an equality strong enough to identify the frozen product with the comb, yet weak enough to remain decidable. Full extensionality would identify them, but it is undecidable and identifies far more than arithmetic requires.

5. R_arith: a local equality that carries arithmetic

5.1 The two boxes

On weak normal forms, products and sums have fixed shapes. Section 4.2 exhibited the product box: $S(K\,\bar{m})\,\bar{n}$ behaves pointwise as m · n. There is likewise a sum box: the term $S(S(KB)\,\bar{a})\,\bar{b}$ satisfies, pointwise,

```
S(S(KB) ā) b̄ f  ->w  (S(KB) ā f)(b̄ f)  ->w  (B (ā f))(b̄ f)
```

and $B\,(\bar{a}\,f)(\bar{b}\,f)\,z \longrightarrow_w (\bar{a}\,f)((\bar{b}\,f)\,z) = f^{a(f}b\,z) = f^{\wedge}(a+b)\,z$. So $S(S(KB)\,\bar{a})\,\bar{b}$ is 'a + b in suspended form'.

5.2 The four rules

The question of Section 4 becomes: which local rewrites turn boxes into combs? The answer is not an η-style patch but the Peano recursions themselves, read as tree rewrites on {K, S}-normal forms. Writing u, v for arbitrary subterms, R_arith consists of:

```
R1   S(K(SB u)) v        ->   S(S(KB) v)(S(K u) v)        (u+1) * v  =  v + u * v
R2   S(K(KI)) v          ->   KI                           0 * v      =  0
R3   S(S(KB)(SB u)) v    ->   SB (S(S(KB) u) v)            (u+1) + v  =  1 + (u +
v)
R4   S(S(KB)(KI)) v      ->   v                            0 + v      =  v
```

Each rule is pointwise weakly valid: applying both sides to one fresh variable (for R1, R2) or two fresh variables (for R3, R4) and weak-normalizing yields syntactically identical results. This was machine-checked for all four rules and is the exact sense in which the rules 'do not change the number'. Section 7 upgrades these checks to theorems quantified over arbitrary parameter terms.

Rule R3 deserves a second look. At the level of the functions being iterated it says $(f \circ (u\,f)) \circ (v\,f) = f \circ ((u\,f) \circ (v\,f))$: it is the associativity of composition. The rule that World One needed, associativity, has returned, now as the workhorse of the system.

5.3 The main computation

The frozen product of Section 4 now evaluates. Starting from S(K 2) 3 and applying only R-rules (no weak steps are needed; every intermediate term is already weak-normal), the run in arithmetic reading is:

```
2*3  =>  3 + (1*3)  =>  3 + (3 + (0*3))  =>  3 + (3 + 0)  =>  3 + 3  =>  6
```

and the final term is syntactically the comb 6. The run takes 11 rule applications: R1 twice, R2 once, R3 six times, and R4 twice. More generally, machine checks confirm B m̄ n̄ →wR (m·n)‾ for all m, n ≤ 7, and the flat-box step count is exactly

```
steps(m, n)  =  m*n + 2m + 1        (R1: m times, R2: once, R3: m*n times, R4: m
times)
```

The count is asymmetric: evaluating 2 ⊗ 3 costs 11 steps while 3 ⊗ 2 costs 13, although both end at 6. The tree-level non-commutativity of Section 3.1 survives inside the arithmetic as a cost asymmetry, even after the results are identified.

5.4 Emergence, confluence, termination

Commutativity and associativity are stated by none of the four rules, yet both hold as equalities of normal forms: NF(B 4̄ 5̄) = NF(B 5̄ 4̄) = 20̄, and NF(B(B 2̄ 3̄) 4̄) = NF(B 2̄ (B 3̄ 4̄)) = 24̄ (machine-checked). The commutative monoid of multiplication is emergent.

Local confluence holds by a critical-pair analysis. The four R-rules are left-linear and their left-hand sides are pairwise non-unifiable (the boxed first arguments K(SB u), K(KI), S(KB)(SB u), and S(KB)(KI) have distinct rigid heads). The only overlaps with the weak rules arise when an R-redex receives one further argument, so that the root simultaneously becomes an S-redex; each such critical pair converges, and the convergence computation is exactly the pointwise-validity computation already verified for each rule. Randomized experiments support global confluence: 60 nested product expressions, each reduced under four independently randomized redex-selection orders, always reached the same normal form.

Termination is established in exact form for flat boxes (the step-count formula above; sums cost a + 1 steps), and experimentally for nested expressions: no run in any experiment, randomized or not, failed to terminate. A fully general strong-normalization proof, for instance by a polynomial interpretation, is open; the subtlety is that R1 duplicates its argument v, so naive decreasing measures fail on deeply nested expressions. We flag this honestly as a proof obligation rather than a result.

5.5 The equality =wR

Define =wR as the convertibility generated by weak reduction together with R_arith. On the fragment where confluence and termination are established, =wR is decidable: normalize both sides and compare trees. Three facts locate it precisely on the ladder of Section 2.3. It is strictly coarser than =w, since it identifies S(K 2)3 with 6. It carries all of numeral arithmetic, as Sections 5.3 and 5.4 show. And it is strictly finer than =ext: the η-instance S(Kx)I, which is extensionally equal to x, is a stuck =wR-normal form distinct from x (machine-checked). So the ladder reads

```
syntactic identity  ⊊  =w  ⊊  =wR  ⊊  =ext
```

and arithmetic lives exactly at =wR. Full extensionality is not needed for numbers. This is the paper's answer to the project's central design question.

6. Primality as unreachability

6.1 Inverting the system

R_arith is directed: it evaluates, from visible products toward combs. Factoring is therefore running it backward. A term t is called backward-reachable from n̄ if t $\longrightarrow^{*}_{wR}$ n̄, that is, if some forward reduction carries t to the comb. The factorization question becomes: is any product box S(K m̄)‑k with m, k ≥ 2 backward-reachable from n̄?

We implemented the four inverse rules under a strict verification discipline: every single backward step is checked by applying the forward rule to its result and confirming that it reproduces the predecessor exactly, and every successful factorization additionally carries a certificate, namely a forward normalization of the found product box back to n̄.

Two inverse rules make the backward space wild. R4 inverted inserts a '0 +' layer around any subterm whatsoever, and R2 inverted rewrites KI to S(K(KI)) v for an arbitrary v: the search space is infinitely branching. This wildness is not a defect; it is the reason the confluence result of Section 5 matters, as Section 6.4 explains.

6.2 The canonical stride-k strategy

There is nonetheless a canonical way to search, indexed by a stride k with 2 ≤ k ≤ n–1. Repeatedly cut k links off the residual comb: one inverse R4 seeds a '0 +' box at the cut point, and k inverse R3 steps pull k comb links into the box, producing one layer of '+k'. If the residual comb reaches length exactly 0, apply inverse R2 with the single guess v := ‑k, and collapse the resulting tower from the inside out by inverse R1 steps into the product box S(K (n/k)‾)‑k. In arithmetic reading, machine-produced traces look as follows:

```
n=6, k=2:   6  ~>  2+4  ~>  2+(2+2)  ~>  2+(2+(2+0))
              ~>  2+(2+(2+(0*2)))  ~>  3*2          SUCCESS (13 steps, certified)
n=7, k=2:   7  ~>  2+5  ~>  2+(2+3)  ~>  2+(2+(2+1))   DEAD END (residue 1)
```

The successful run is, step for step, the forward evaluation of Section 5.3 read in reverse, and its length 13 equals the forward cost of 3 · 2.

6.3 Dead-end anatomy and residue geometry

When n mod k = r with 0 < r < k, the stride-k search strands on a residual comb r̄ that is blocked in three independent ways. Inverse R2 requires the residual to be exactly KI, and r̄ is not. The next cut requires k comb links, and r̄ has only r < k. And inverse R1 requires the addition slot of the innermost layer to be exactly ‑k, which no further move can arrange. The stranded term for n = 7, k = 2 is, written out in full,

```
S(S(KB) 2̄)(S(S(KB) 2̄)(S(S(KB) 2̄)(1̄)))
```

a tower of three '+2' layers over the too-short comb 1̄.

Running all strides produces, for each n, a residue vector whose entries are the lengths of the terminal residual combs; a zero marks a found factor. Machine output for six values:

n	residue vector (k = 2 … n–1)	verdict
6	0 0 2 1	composite
7	1 1 3 2 1	prime
11	1 2 3 1 5 4 3 2 1	prime
12	0 0 0 2 0 5 4 3 2 1	composite
13	1 1 1 3 1 6 5 4 3 2 1	prime

15	1 0 3 0 3 1 7 6 5 4 3 2 1	composite

The Euclidean residues n mod k thus appear not as computed quantities but as the geometry of terminal dead-end shapes: a prime is a number all of whose stride searches strand on a nonempty stub.

6.4 From non-observation to a theorem: the role of confluence

The canonical strategy might in principle miss factorizations that a wilder backward search would find, and the wild search space is infinite, so no amount of searching could ever establish primality empirically. Bounded exhaustive searches illustrate both points. With a size bound of roughly twice the numeral, the entire backward-reachable space of 4 comprises 20,911 distinct terms and contains exactly one product box, namely 2 · 2, found at distance 9, which is precisely the forward evaluation cost of 2 ⊗ 2. The entire bounded space of 5 comprises 99,012 distinct terms and contains no product box at all. For 6 the bounded space already exceeds the memory of the experiment: the combinatorial wall is real.

Confluence removes the wall. Suppose some product box S(K m̄)k̄ with m, k ≥ 2 satisfies S(K m̄)k̄ ⟶*wR n̄. The forward evaluation of Section 5.3 gives S(K m̄)k̄* ⟶wR (m·k)‾, by a derivation that is uniform in m and k (machine-checked for all m, k ≤ 7). Both n̄ and (m·k)‾ are =wR-normal forms of the same term; by confluence on this class they coincide, so m · k = n. This yields:

Theorem B. For n ≥ 2, and granting confluence of w + R_arith on arithmetic expressions (Section 5.4): n is prime if and only if no term S(K m̄)k̄ with m, k ≥ 2 is backward-reachable from n̄. Composite witnesses are produced constructively and certified by the canonical strategy.

The conceptual point deserves emphasis, because it is the payoff of the project's founding intuition. Primality is here a negative existential over an infinite space of trees, and no search can prove it. What proves it is a property of the equality itself: confluence converts 'never observed' into 'unreachable'. The load-bearing element of the number system is not the trees but the metatheory of the identification imposed on them.

6.5 Costs, and the free-monoid reading

A stride-k probe costs about n rewrite steps (n – r inverse R3 steps plus one bookkeeping step per layer), so a full primality certificate for n costs the sum over all strides: 35 steps for n = 7, 94 for n = 11, and 138 for n = 13, a quadratic growth. This is exactly what a unary encoding deserves. The canonical numerals form the free monoid on the single link SB; multiplication is exponent multiplication there, so factoring n̄ means recognizing the word $(SB)^n$ as a proper power, and a prime is an aperiodic comb. In a unary alphabet, periodicity cannot be seen locally; it must be probed stride by stride, and each probe must traverse the whole word. The rewriting system does not hide this; it makes it precise.

7. From verified runs to verified proofs

Every computational claim in this paper was machine-checked in the sense of Appendix A: an engine computed the instances and printed its findings. This section reports a stronger fact, established after version 0.2 and shared with the companion paper [15]: the paper's central schematic results are now theorems inside Idris2' s type system. Statements are types, proofs are total terms, and the compiler checks both once. The module Proofs.idr, grown to 3,337 lines since this paper's first pass and shared by every paper of the series, is mapped in Appendix C; it is declared %default total, contains no postulates and no holes, and the single command idris2 —check Proofs.idr is the entire verification.

The change is one of authority. Weak reduction lives in the module not as an evaluator but as an inductive relation, one constructor per contraction and two for closure under application contexts, with its reflexive transitive closure on top:

```
data Step : Term -> Term -> Type where
  RedK : Step (App (App CK a) b) a
  RedS : Step (App (App (App CS a) b) c) (App (App a c) (App b c))
  CtxL : Step l l' -> Step (App l r) (App l' r)
  CtxR : Step r r' -> Step (App l r) (App l r')

data Steps : Term -> Term -> Type where
  Done : Steps t t
  Then : Step s t -> Steps t u -> Steps s u
```

On this bedrock sit three pieces of infrastructure: congruence lifts and transitivity for assembling long derivations from short ones, a substitution lemma (variables take no step, so every derivation at generic parameters is stable under substitution), and a certified evaluator wnfCert : Nat -> (t : Term) -> (t' ** Steps t t') that returns each weak normal form together with its derivation, demoting the engine of Appendix B from arbiter to an oracle whose answers carry proofs.

Three results of this paper are covered.

Theorem A, end to end. The multiplicativity of grafting, stated as immediate in Section 3.1 and machine-checked on instances, is a compiled induction:

```
leavesGraft : (s, t : LTree) ->
  leaves (graft s t) = leaves s * leaves t
```

On top of it, Theorem A is formalized constructively as an equivalence between composite sizes and decomposable trees, theoremAto and theoremAfrom, with both directions producing witnesses: a decomposition yields a factorization of the leaf count, and a factorization yields explicit trees whose graft has the required size. The prime formulation of Section 3.3 is recovered as corollaries in both directions from PrimeN n, defined as n at least 2 together with the absence of a factorization. The census of Section 3.3 verified sizes to twelve leaves; the theorem now covers every size, with no appeal to decidability anywhere.

Adequacy of the numerals. The fact used throughout Section 4, that the numeral of n applied to f and x weakly reduces to the n-fold application, is proved by induction on n, the successor case riding on a certified four-step derivation of the B combinator's action:

```
numAdequate : (n : Nat) -> (x : Term) ->
  Steps (App (App (num n) (Var "f")) x) (fpow n x)
```

Validity of R1 to R4, strengthened. Section 5.2 checked pointwise validity at fresh variables. The module proves more: for every rule and all parameter terms u and v whatsoever, the two sides applied to f and z are joinable, with the common reduct exhibited:

```
r1Valid : (u, v : Term) ->
  Joinable (App (App (r1L u v) (Var "f")) (Var "z"))
           (App (App (r1R u v) (Var "f")) (Var "z"))
```

The fresh-variable checks of Section 5.2 are the special case in which u and v are variables; the theorems quantify over arbitrary terms, which is exactly the license needed when the rules fire inside larger expressions.

Theorem B's fragment form, delivered without confluence. Version 0.4 discharges the second-tier item announced in version 0.3, and by a shorter route than Section 6.4′ s. R_arith lives in the module as an inductive relation StepR, one constructor per rule and two for closure under application contexts. The schematic forward derivation is a five-line induction,

```
boxReduces : (m, k : Nat) ->
  StepsR (prodBox (num m) (num k)) (num (m * k))
```

and the backward direction is a step-preserved value invariant with no appeal to confluence at all: arithmetic forms are an inductive family AF : Nat -> Term -> Type indexed by their value, R steps preserve the index (stepRAF), and the index at a numeral is unique (afUnique). Together these yield the equivalence: a product box built from m̄ and k̄, both at least 2, R-reduces to n̄ exactly when n is the product (theoremBto, theoremBfrom), with the primality corollaries in both directions. Section 6.4′ s confluence framing remains the right conceptual account of why non-observation was already evidence; the invariant supplies the shorter formal proof, and the fragment restriction, product boxes of numerals rather than arbitrary nested expressions, is the honest boundary of what is proved. Strong normalization of R_arith is untouched by any of this and remains the open obligation recorded in Section 9. Local confluence by critical pairs, and the Church-Rosser property for weak reduction, sit in the third tier. The toolchain is Idris2 0.8.0 [16] with the base library only; the same module also carries the companion paper's theorems [15], since both papers share one calculus, including its divisibility probe correctness in divmod form.

8. Relation to other work

Three bodies of work border this project: numeral systems in the lambda calculus and combinatory logic, arithmetic presented as term rewriting, and arithmetics defined directly on trees.

Numeral systems and equalities. The iterator numerals of Section 4 are the classical Church numerals transported to combinatory logic [1, 2, 14]. The numeral-systems literature, surveyed by Barendregt [5], fixes an equality, usually beta or weak conversion, and asks which numeral sequences are adequate, that is, admit definable successor, predecessor, and zero test. This paper varies the other parameter. The numerals stay standard and the equality moves: weak conversion is too fine (Section 4), extensional equality is undecidable (Section 2.3), and =wR is placed strictly between them so that arithmetic becomes total and decidable. Treating the equality itself as the design variable, and locating arithmetic at an intermediate decidable congruence, is the departure point from that literature. Curry's finitely axiomatized extensional theories [1] remain the natural comparison targets for =wR; Section 9 records this as open.

Arithmetic as term rewriting. Rules R1 to R4 are the Peano recursions for addition and multiplication, oriented as rewrite rules. Over a fresh signature with zero and successor, this system is the standard introductory example of term rewriting, and its confluence and termination are textbook results [6]; the critical-pair method used in Section 5.4 goes back to Knuth and Bendix [7], and Newman's lemma supplies the bridge from local to global confluence [6]. The difference here is the carrier. R_arith acts inside the applicative {K, S} syntax, on weak normal forms that the reduction theory itself produces, and the numerals, sums, and products are not fresh constructors but frozen boxes that weak reduction cannot open. This is also where the classical results stop transferring: rule R1 duplicates its right argument, the textbook termination measure fails on nested boxes, and strong normalization for the full system is the open obligation recorded in Section 9.

Trees as numbers. Encoding numbers as nested structures is old: Ackermann's bijection between hereditarily finite sets and the naturals [12] is the ancestor of the genre, and Tarau has developed tree-represented numeral systems whose arithmetic operates directly on the structure, including hereditarily binary numbers with compact representations [4]. Those systems choose the encod-

ing so that arithmetic becomes structural by design. World Two runs the opposite experiment: the encoding is the standard iterator, and the shapes that end up carrying arithmetic, the combs and the two boxes, are forced by the reduction theory rather than chosen.

Tree arithmetics and prime trees. Closest to World One is the line opened by Loday's arithmetree [8], which defines addition and multiplication on planar binary trees extending those of the natural numbers; Bruno and Yasaki study primality inside that system [9], and Luccio builds an independent arithmetic on rooted unordered trees with a polynomial-time primality test and unique factorization [10]. Sizes multiply under all these products, so a tree of prime size is automatically prime there, just as Theorem A makes every tree of prime leaf count indecomposable here; results of this kind are elementary in each setting. The census is where the perspectives differ. In those systems primality is a property of an individual tree, and the interesting objects are the prime trees living at composite sizes. Here the object is the size class itself: the census quantifies how rare decomposable trees are against the Catalan background (98 out of 58,786 at twelve leaves), and World One earns its place in this project not through Theorem A but by identifying associativity as the equality that first turns trees into numbers. Grafting, substitution at every leaf, also differs from the root-merging and leaf-attaching products of [8, 9, 10].

Primality from computation. Conway's FRACTRAN [11] made the primes the output trace of an unbounded rewriting process; there, primality appears as reachability along the run of a universal machine, and nothing is decidable by inspection. Theorem B is a reachability statement with the quantifier tamed: because the fragment is confluent and terminating, unreachability over the infinite backward space reduces to a finite certified check, and the exhaustive searches of Section 6 become proofs rather than evidence. The free-monoid reading of Section 6.5, primes as aperiodic combs, echoes primitive words in combinatorics on words [13]: over a one-letter alphabet, a word is a proper power exactly when its length is composite, and the comb encoding realizes this inside {K, S}. What the rewriting frame adds to that elementary fact is the dead-end anatomy: the residues n mod k appear as the geometric shapes at which every stride-k strategy strands.

To our knowledge, no prior tree arithmetic works modulo a reduction-generated equality on combinators, which is the specific mechanism studied here.

9. Conclusion: how far we have come

Established. (1) In the world of bare trees, a complete structural characterization of primality: n is prime exactly when the entire Catalan family of n-leaf trees is free of uniform cut lines (Theorem A), with a full census for $n \le 12$; and the insight that raw trees are almost all 'prime', arithmetic emerging only through the quotient by associativity. (2) A concrete combinatory number system in which numbers are equivalence classes of {K, S}-trees, numerals are right combs over the link SB, addition of canonical numerals is literal concatenation, and multiplication is the combinator B, whose weak reduction freezes into a product box that keeps both factors visible. (3) The four-rule system R_arith, pointwise valid, locally confluent by critical-pair analysis, terminating with exact step counts on flat boxes, under which the product box of 2 and 3 rewrites to the comb 6 in 11 steps, all arithmetic identities of the tested range hold, and commutativity and associativity are emergent. (4) The graded ladder of equalities, syntactic identity $\subsetneq$ =w $\subsetneq$ =wR $\subsetneq$ =ext, with the proof that arithmetic lives exactly at the decidable level =wR and that full extensionality is unnecessary for numbers. (5) The reformulation of primality as certified unreachability (Theorem B), together with the dead-end anatomy: residues as terminal tree shapes, exhaustively confirmed in bounded backward spaces of up to 99,012 terms, and the identification of confluence as the proof engine that makes the negative existential provable.

Validated thesis. The project began from the conviction that the equality concept would be the decisive element of a tree-based number system. The arc of results bears this out three times over:

associativity alone creates the numbers in World One; the placement of =wR between weak and extensional equality is exactly what makes arithmetic both total and decidable in World Two; and the confluence of =wR, a property of the equality rather than of any tree, is what turns primality into a theorem.

Not yet achieved. Honesty requires an equally clear list. First, the structural account of primality obtained so far re-expresses trial division: the residue vector is classical, and the novelty lies in its appearance as dead-end geometry and in the unreachability framing, not in any new arithmetic criterion. Second, the unary comb encoding makes divisibility inherently non-local and the costs quadratic; the early warning that the encoding is not neutral now has teeth. The companion paper [15] takes this up directly, with binary numerals and divisibility probes whose costs are logarithmic in n. Third, unique factorization is now formulated inside the system by the companion paper [15], as confluence of a dedicated factoring rewrite; discharging its arithmetic hypothesis inside the type system remains open there. Fourth, the relation of =wR to the finitely axiomatized extensional theories of Curry remains unexplored, and η lies deliberately outside =wR. Fifth, while Theorem A, the numeral adequacy, the universal validity of all four rules, and the fragment form of Theorem B are now compiler-proved (Section 7), the confluence and termination arguments for nested expressions remain paper-level, with the duplication in rule R1 identified as the obstacle to a naive termination measure; full strong normalization is an open proof obligation.

Next steps. The most promising continuation attacks the encoding: replace unary combs by genuinely binary tree numerals, built from doubling and successor combinators, and re-run the entire program, asking whether divisibility becomes more local, how the dead-end anatomy transforms, and whether the tree shape itself can carry arithmetic information at sublinear probe cost. In parallel: formulate and prove unique factorization as a rewriting statement; complete the strong-normalization proof; and connect the construction to the literature on tree-represented arithmetic and on combinatory extensionality.

The distance covered can be summarized in one sentence: we set out to make 'prime' a property of trees, and arrived at 'prime' as a provable property of an equality on trees, decidable, machine-checked, and with its classical content, the Euclidean residues, laid bare as the shapes at which a rewriting system strands.

Appendix A: Methods

All experiments ran on a small purpose-built rewriting engine (Python, roughly 200 lines per experiment). Terms are binary trees over the leaves {K, S, variables}; reduction uses leftmost-outermost redex selection unless randomized selection is stated. The Python implementation of the exhaustive backward searches uses hash-consing, giving constant-time equality, hashing, and size queries; the Idris2 port reaches the same full exhaustion of spaces with about 100,000 terms using a plain ordered set over structural comparison. The verification discipline is uniform: every claimed reduction is executed, not asserted; every inverse step is validated by running the corresponding forward rule on its output and comparing with the input; every factorization found is certified by forward normalization to the target numeral; and pointwise validity of rules is checked by applying both sides to fresh variables and comparing weak normal forms. The four experiment scripts (census and grafting; R_arith; canonical backward strategy; exhaustive backward search) are available alongside this paper, both as the original Python scripts and as a consolidated Idris2 port (Secret1.idr, compiled with Idris2 0.8.0 on the Chez Scheme backend) that reproduces every reported figure.

The proof module of Section 7, Proofs.idr, is the series' shared artifact, mapped in Appendix C and checked with idris2 —check Proofs.idr; it is total by declaration, imports only the base library, and contains no postulates or holes, so a clean check is the complete verification.

Appendix B: Idris2 source

The consolidated Idris2 port referenced in Appendix A, verbatim. It compiles with Idris2 0.8.0 on the Chez Scheme backend and runs with idris2 Secret1.idr -x main, printing every figure reported in Sections 3 to 6.

```
||| Project secret1 : a tree-based number system over the combinators K and S.
||| Idris2 port of the four Python experiment scripts accompanying the
||| working paper (census/grafting, R_arith, backward strategy, bounded BFS).
||| Run with:  idris2 Secret1.idr -x main
module Secret1

import Data.List
import Data.Maybe
import Data.String
import Data.SortedMap
import Data.SortedSet

%default covering

--------------------------------------------------------------------------
-- helpers
--------------------------------------------------------------------------

||| ascending [lo .. hi], empty when lo > hi
upTo : Nat -> Nat -> List Nat
upTo lo hi = map (lo +) (below (minus (S hi) lo))
  where
    below : Nat -> List Nat
    below Z     = []
    below (S k) = below k ++ [k]

boolMark : Bool -> String
boolMark True  = "OK"
boolMark False = "FAIL"

divN : Nat -> Nat -> Nat
divN n k = fromInteger (div (cast n) (cast k))

modN : Nat -> Nat -> Nat
modN n k = fromInteger (mod (cast n) (cast k))

--------------------------------------------------------------------------
-- World One : bare leaf trees, grafting, census
--------------------------------------------------------------------------

data LTree = Leaf | LNode LTree LTree

Eq LTree where
  Leaf      == Leaf      = True
  LNode a b == LNode c d = a == c && b == d
  _         == _         = False

showL : LTree -> String
showL Leaf        = "."
showL (LNode l r) = "(" ++ showL l ++ showL r ++ ")"
```

```
||| replace every leaf of s by a copy of t
graft : LTree -> LTree -> LTree
graft Leaf        t = t
graft (LNode l r) t = LNode (graft l t) (graft r t)

||| table: n |-> all trees with n leaves
treeTable : Nat -> SortedMap Nat (List LTree)
treeTable top = foldl step SortedMap.empty (upTo 1 top)
  where
    row : SortedMap Nat (List LTree) -> Nat -> List LTree
    row m n = concatMap (\k =>
                let ls = fromMaybe [] (SortedMap.lookup k m)
                    rs = fromMaybe [] (SortedMap.lookup (minus n k) m)
                in concatMap (\a => map (LNode a) rs) ls)
              (upTo 1 (minus n 1))
    step : SortedMap Nat (List LTree) -> Nat -> SortedMap Nat (List LTree)
    step m 1 = SortedMap.insert 1 [Leaf] m
    step m n = SortedMap.insert n (row m n) m

||| number of distinct trees with n leaves of the form s (x) t, both factors >= 2
decomposables : SortedMap Nat (List LTree) -> Nat -> Nat
decomposables tab n = length (nub cands)
  where
    build : Nat -> List LTree
    build a =
      let ss = fromMaybe [] (SortedMap.lookup a tab)
          ts = fromMaybe [] (SortedMap.lookup (divN n a) tab)
      in concatMap (\s => map (graft s) ts) ss
    cands : List LTree
    cands = concatMap build (filter (\a => modN n a == 0) (upTo 2 (minus n 1)))

-----------------------------------------------------------------------
-- World Two : combinatory terms
-----------------------------------------------------------------------

data Term = CS | CK | Var String | App Term Term

Eq Term where
  CS      == CS      = True
  CK      == CK      = True
  Var a   == Var b   = a == b
  App a b == App c d = a == c && b == d
  _       == _       = False

Ord Term where
  compare CS CS           = EQ
  compare CS _            = LT
  compare _  CS           = GT
  compare CK CK           = EQ
  compare CK _            = LT
  compare _  CK           = GT
  compare (Var a) (Var b) = compare a b
  compare (Var _) _       = LT
  compare _ (Var _)       = GT
```

```
  compare (App a b) (App c d) =
    case compare a c of
      EQ => compare b d
      o  => o

Show Term where
  show CS      = "S"
  show CK      = "K"
  show (Var v) = v
  show (App l r) =
    show l ++ (case r of
                 App _ _ => "(" ++ show r ++ ")"
                 _       => show r)

iC, bC, ki, sb, skb : Term
iC  = App (App CS CK) CK                -- I   = SKK
bC  = App (App CS (App CK CS)) CK       -- B   = S(KS)K
ki  = App CK iC                         -- KI  = zero
sb  = App CS bC                         -- SB  = successor link
skb = App CS (App CK bC)                -- S(KB), head of the sum box

num : Nat -> Term
num Z     = ki
num (S n) = App sb (num n)

prodBox : Term -> Term -> Term          -- S(K m) k    ("m * k")
prodBox m k = App (App CS (App CK m)) k

multB : Term -> Term -> Term            -- B m n
multB m n = App (App bC m) n

-----------------------------------------------------------------
-- forward rules : weak reduction + R_arith
-----------------------------------------------------------------

matchRoot : Term -> Maybe (String, Term)
matchRoot (App (App CK a) _) = Just ("wK", a)
matchRoot (App (App (App CS a) b) c) =
  Just ("wS", App (App a c) (App b c))
matchRoot (App (App CS (App CK y)) v) =
  if y == ki
    then Just ("R2", ki)
    else case y of
           App h u => if h == sb
                        then Just ("R1",
                               App (App CS (App skb v))
                                   (App (App CS (App CK u)) v))
                        else Nothing
           _ => Nothing
matchRoot (App (App CS (App h z)) v) =
  if h == skb
    then if z == ki
           then Just ("R4", v)
           else case z of
                  App h2 u => if h2 == sb
```

```
                                then Just ("R3",
                                       App sb (App (App CS (App skb u)) v))
                                else Nothing
                  _ => Nothing
    else Nothing
matchRoot _ = Nothing

stepNamed : Term -> Maybe (String, Term)
stepNamed t =
  case matchRoot t of
    Just r  => Just r
    Nothing =>
      case t of
        App l r =>
          case stepNamed l of
            Just (nm, l') => Just (nm, App l' r)
            Nothing =>
              case stepNamed r of
                Just (nm, r') => Just (nm, App l r')
                Nothing       => Nothing
        _ => Nothing

Counts : Type
Counts = SortedMap String Nat

bump : String -> Counts -> Counts
bump nm c = SortedMap.insert nm (S (fromMaybe 0 (SortedMap.lookup nm c))) c

nfCount : Nat -> Term -> Maybe (Term, Counts)
nfCount fuel t = go fuel t SortedMap.empty
  where
    go : Nat -> Term -> Counts -> Maybe (Term, Counts)
    go Z     _ _ = Nothing
    go (S f) u c =
      case stepNamed u of
        Nothing       => Just (u, c)
        Just (nm, u') => go f u' (bump nm c)

nf : Term -> Maybe Term
nf t = map fst (nfCount 200000 t)

-- weak-only reduction, for the pointwise validity checks
matchW : Term -> Maybe Term
matchW (App (App CK a) _)         = Just a
matchW (App (App (App CS a) b) c) = Just (App (App a c) (App b c))
matchW _                          = Nothing

stepW : Term -> Maybe Term
stepW t =
  case matchW t of
    Just r  => Just r
    Nothing =>
      case t of
        App l r =>
          case stepW l of
```

```
            Just l' => Just (App l' r)
            Nothing => map (App l) (stepW r)
        _ => Nothing

wnf : Term -> Term
wnf = go 200000
  where
    go : Nat -> Term -> Term
    go Z     t = t
    go (S f) t = case stepW t of
                   Nothing => t
                   Just t' => go f t'

ruleValid : (Term, Term) -> Bool
ruleValid (lhs, rhs) =
  let f = Var "f"
      z = Var "z"
  in wnf (App (App lhs f) z) == wnf (App (App rhs f) z)

r1L, r1R, r2L, r2R, r3L, r3R, r4L, r4R : Term
r1L = App (App CS (App CK (App sb (Var "u")))) (Var "v")
r1R = App (App CS (App skb (Var "v"))) (App (App CS (App CK (Var "u"))) (Var "v"))
r2L = App (App CS (App CK ki)) (Var "v")
r2R = ki
r3L = App (App CS (App skb (App sb (Var "u")))) (Var "v")
r3R = App sb (App (App CS (App skb (Var "u"))) (Var "v"))
r4L = App (App CS (App skb ki)) (Var "v")
r4R = Var "v"

-----------------------------------------------------------------------
-- positions, inverse rules (each step verified by the forward rule)
-----------------------------------------------------------------------

data Dir = DL | DR

getAt : Term -> List Dir -> Maybe Term
getAt t          []        = Just t
getAt (App l _) (DL :: p)  = getAt l p
getAt (App _ r) (DR :: p)  = getAt r p
getAt _          _         = Nothing

putAt : Term -> List Dir -> Term -> Maybe Term
putAt _         []        new = Just new
putAt (App l r) (DL :: p) new = map (\l' => App l' r) (putAt l p new)
putAt (App l r) (DR :: p) new = map (App l)           (putAt r p new)
putAt _         _         _   = Nothing

||| insert newSub at path only if the forward rule named nm maps it
||| back exactly onto the current subterm
applyInv : Term -> List Dir -> String -> Term -> Maybe Term
applyInv t path nm newSub = do
  old        <- getAt t path
  (nm', red) <- matchRoot newSub
  if nm' == nm && red == old
    then putAt t path newSub
```

```
    else Nothing

invR4 : Term -> List Dir -> Maybe Term
invR4 t p = do
  v <- getAt t p
  applyInv t p "R4" (App (App CS (App skb ki)) v)

invR3 : Term -> List Dir -> Maybe Term
invR3 t p = do
  sub <- getAt t p
  case sub of
    App h (App (App CS (App h2 u)) v) =>
      if h == sb && h2 == skb
        then applyInv t p "R3" (App (App CS (App skb (App sb u))) v)
        else Nothing
    _ => Nothing

invR2 : Term -> List Dir -> Term -> Maybe Term
invR2 t p v = do
  old <- getAt t p
  if old == ki
    then applyInv t p "R2" (App (App CS (App CK ki)) v)
    else Nothing

invR1 : Term -> List Dir -> Maybe Term
invR1 t p = do
  sub <- getAt t p
  case sub of
    App (App CS (App h1 v1)) (App (App CS (App CK u)) v2) =>
      if h1 == skb && v1 == v2
        then applyInv t p "R1" (App (App CS (App CK (App sb u))) v1)
        else Nothing
    _ => Nothing

-----------------------------------------------------------------------
-- arithmetic reading of a state
-----------------------------------------------------------------------

combLen : Term -> Maybe Nat
combLen t =
  if t == ki
    then Just Z
    else case t of
           App h u => if h == sb then map S (combLen u) else Nothing
           _       => Nothing

peel : Term -> (List Nat, Term)
peel t@(App (App CS (App h aT)) rest) =
  case (h == skb, combLen aT) of
    (True, Just a) => let (ls, c) = peel rest in (a :: ls, c)
    _              => ([], t)
peel t = ([], t)

describe : Term -> String
describe t =
```

```
  let (layers, c) = peel t
  in fst (foldr wrapLayer (coreS c) layers)
  where
    coreS : Term -> (String, Bool)
    coreS c =
      case combLen c of
        Just n  => (show n, False)
        Nothing =>
          case c of
            App (App CS (App CK mT)) kT =>
              case (combLen mT, combLen kT) of
                (Just m, Just k) => (show m ++ "*" ++ show k, True)
                _                => ("?", True)
            _ => ("?", True)
    wrapLayer : Nat -> (String, Bool) -> (String, Bool)
    wrapLayer a (s, comp) =
      (show a ++ "+" ++ (if comp then "(" ++ s ++ ")" else s), True)

-------------------------------------------------------------------
-- canonical backward strategy with stride k
-------------------------------------------------------------------

record BackRes where
  constructor MkBack
  bSuccess : Bool
  bRest    : Nat
  bSteps   : Nat
  bFinal   : Term
  bTrace   : List String

rpath : Nat -> List Dir
rpath d = replicate d DR

unlayer : Term -> Nat -> Nat -> Maybe (Term, Nat)
unlayer t Z     st = Just (t, st)
unlayer t (S d) st =
  case invR1 t (rpath d) of
    Just t' => unlayer t' d (S st)
    Nothing => Nothing

r3run : Term -> Nat -> Nat -> Nat -> Maybe (Term, Nat)
r3run t depth Z     st = Just (t, st)
r3run t depth (S j) st =
  case invR3 t (rpath (depth + j)) of
    Just t' => r3run t' depth j (S st)
    Nothing => Nothing

backward : Nat -> Nat -> BackRes
backward n k = loop (num n) Z Z [describe (num n)] (S (n + n + 10))
  where
    giveUp : Nat -> Term -> Nat -> List String -> BackRes
    giveUp r t st tr = MkBack False r st t (reverse tr)
    loop : Term -> Nat -> Nat -> List String -> Nat -> BackRes
    loop t depth st tr Z = giveUp 999 t st tr
    loop t depth st tr (S fuel) =
```

```
      case getAt t (rpath depth) of
        Nothing => giveUp 998 t st tr
        Just r  =>
          case combLen r of
            Nothing  => giveUp 997 t st tr
            Just len =>
              if len == Z then
                case invR2 t (rpath depth) (num k) of
                  Nothing => giveUp 996 t st tr
                  Just t1 =>
                    case unlayer t1 depth (S st) of
                      Nothing        => giveUp 995 t1 (S st) (describe t1 :: tr)
                      Just (t2, st2) =>
                        MkBack True Z st2 t2
                               (reverse (describe t2 :: describe t1 :: tr))
              else if len >= k then
                case invR4 t (rpath (depth + k)) of
                  Nothing => giveUp 994 t st tr
                  Just t1 =>
                    case r3run t1 depth k (S st) of
                      Nothing        => giveUp 993 t1 (S st) tr
                      Just (t2, st2) =>
                        loop t2 (S depth) st2 (describe t2 :: tr) fuel
              else giveUp len t st tr

certified : Nat -> Nat -> BackRes -> Bool
certified n k r =
  r.bSuccess
  && r.bFinal == prodBox (num (divN n k)) (num k)
  && nf r.bFinal == Just (num n)

residueRow : Nat -> (List Nat, Nat)
residueRow n =
  let rs = map (backward n) (upTo 2 (minus n 1))
  in ( map (\r => if r.bSuccess then Z else r.bRest) rs
     , sum (map (\r => r.bSteps) rs) )

-----------------------------------------------------------------------
-- bounded exhaustive backward search (free BFS)
-----------------------------------------------------------------------

tsize : Term -> Nat
tsize (App l r) = tsize l + tsize r
tsize _         = 1

isProduct : Term -> Maybe (Nat, Nat)
isProduct (App (App CS (App CK mT)) kT) =
  case (combLen mT, combLen kT) of
    (Just m, Just k) => if m >= 2 && k >= 2 then Just (m, k) else Nothing
    _                => Nothing
isProduct _ = Nothing

positions : Term -> List (List Dir)
positions (App l r) =
  [] :: (map (DL ::) (positions l) ++ map (DR ::) (positions r))
```

```
positions _ = [[]]

invMoves : Nat -> Nat -> Term -> List Term
invMoves nmax cap t =
  filter (\u => tsize u <= cap) (mapMaybe id (concatMap at (positions t)))
  where
    at : List Dir -> List (Maybe Term)
    at p =
      case getAt t p of
        Nothing  => []
        Just sub =>
             [invR3 t p, invR1 t p]
          ++ (case combLen sub of
                Just _  => [invR4 t p]
                Nothing => [])
          ++ (if sub == ki
                then map (\v => invR2 t p (num v)) (upTo 2 nmax)
                else [])

bfsRun : Nat -> Nat -> (List ((Nat, Nat), Nat), Nat, Bool)
bfsRun n nodeCap = go initSeen [start] 1 [] 0 nodeCap
  where
    start : Term
    start = num n
    cap : Nat
    cap = 2 * tsize start + 24
    ins : (SortedSet Term, List Term, Nat) -> Term
       -> (SortedSet Term, List Term, Nat)
    ins (se, nx, c) child =
      if SortedSet.contains child se
        then (se, nx, c)
        else (SortedSet.insert child se, child :: nx, S c)
    step : Nat
        -> (SortedSet Term, List Term, Nat, List ((Nat, Nat), Nat))
        -> Term
        -> (SortedSet Term, List Term, Nat, List ((Nat, Nat), Nat))
    step d (se, nx, c, fo) cur =
      case isProduct cur of
        Just pf => (se, nx, c, (pf, d) :: fo)
        Nothing =>
          let (se', nx', c') = foldl ins (se, nx, c) (invMoves n cap cur)
          in (se', nx', c', fo)
    initSeen : SortedSet Term
    initSeen = SortedSet.insert (num n) SortedSet.empty
    go : SortedSet Term -> List Term -> Nat -> List ((Nat, Nat), Nat)
      -> Nat -> Nat -> (List ((Nat, Nat), Nat), Nat, Bool)
    go se frontier c fo d budget =
      case frontier of
        [] => (fo, c, True)
        _  =>
          if budget < length frontier
            then (fo, c, False)
            else
              let (se', nx, c', fo') = foldl (step d) (se, [], c, fo) frontier
              in go se' nx c' fo' (S d) (minus budget (length frontier))
```

```
-- ------------------------------------------------------------------
-- main : run every experiment
-- ------------------------------------------------------------------

main : IO ()
main = do
  putStrLn "== World One: decomposable trees per leaf count =="
  let tab = treeTable 12
  traverse_ (\n => putStrLn $ "  n=" ++ show n ++ ": "
                ++ show (decomposables tab n) ++ " / "
                ++ show (length (fromMaybe [] (SortedMap.lookup n tab))))
            (upTo 2 12)

  putStrLn "\n== grafting is non-commutative on trees =="
  let t2  = LNode Leaf Leaf
  let t3s = fromMaybe [] (SortedMap.lookup 3 tab)
  putStrLn $ "  2(x)3: " ++ unwords (map (\t => showL (graft t2 t)) t3s)
  putStrLn $ "  3(x)2: " ++ unwords (map (\s => showL (graft s t2)) t3s)

  putStrLn "\n== R_arith: pointwise validity of the four rules =="
  traverse_ (\(nm, p) => putStrLn $ "  " ++ nm ++ ": " ++ boolMark (ruleValid p))
    [ ("R1", (r1L, r1R)), ("R2", (r2L, r2R))
    , ("R3", (r3L, r3R)), ("R4", (r4L, r4R)) ]

  putStrLn "\n== frozen product vs comb =="
  putStrLn $ "  wNF(B 2 3) = " ++ show (wnf (multB (num 2) (num 3)))
  putStrLn $ "  equals S(K 2)3: "
           ++ boolMark (wnf (multB (num 2) (num 3)) == prodBox (num 2) (num 3))
  case nfCount 200000 (prodBox (num 2) (num 3)) of
    Just (t, cnt) => do
      putStrLn $ "  wR-NF(S(K 2)3) is the comb 6: " ++ boolMark (t == num 6)
      putStrLn $ "  rule usage: "
        ++ unwords (map (\nm => nm ++ "=" ++ show (fromMaybe 0 (SortedMap.lookup nm
cnt)))
                        ["R1", "R2", "R3", "R4"])
    Nothing => putStrLn "  fuel exhausted"

  putStrLn "\n== multiplication table m,n <= 6 =="
  let pairs = concatMap (\m => map (\n => (m, n)) (upTo 0 6)) (upTo 0 6)
  putStrLn $ "  all B m n = comb(m*n): "
           ++ boolMark (all (\(m, n) =>
                nf (multB (num m) (num n)) == Just (num (m * n))) pairs)

  putStrLn "\n== emergent commutativity / associativity =="
  putStrLn $ "  NF(B 4 5) == NF(B 5 4) == 20: "
    ++ boolMark (nf (multB (num 4) (num 5)) == Just (num 20)
              && nf (multB (num 5) (num 4)) == Just (num 20))
  putStrLn $ "  NF(B(B 2 3)4) == NF(B 2(B 3 4)) == 24: "
    ++ boolMark (nf (multB (multB (num 2) (num 3)) (num 4)) == Just (num 24)
              && nf (multB (num 2) (multB (num 3) (num 4))) == Just (num 24))

  putStrLn "\n== eta stays open below extensionality =="
  putStrLn $ "  wR-NF(S(Kx)I) = "
    ++ (case nf (App (App CS (App CK (Var "x"))) iC) of
```

```
          Just t  => show t
          Nothing => "fuel out")

  putStrLn "\n== backward: canonical stride traces =="
  traverse_ (\(n, k) =>
      let r = backward n k in do
        putStrLn $ "  n=" ++ show n ++ ", k=" ++ show k ++ ":  "
                 ++ concat (intersperse "  ~>  " r.bTrace)
        putStrLn $ "     -> " ++ (if r.bSuccess
                    then "SUCCESS (" ++ show r.bSteps ++ " steps, certified: "
                         ++ boolMark (certified n k r) ++ ")"
                    else "DEAD END rest=" ++ show r.bRest))
    [ (6, 2), (7, 2), (6, 3), (7, 3) ]

  putStrLn "\n== residue vectors (k = 2 .. n-1), 0 = factor found =="
  traverse_ (\n =>
      let (vec, tot) = residueRow n
      in putStrLn $ "  n=" ++ show n ++ ": [" ++ unwords (map show vec) ++ "]  "
                 ++ (if all (/= 0) vec then "PRIME" else "composite")
                 ++ ", steps " ++ show tot)
    [6, 7, 11, 12, 13, 15]

  putStrLn "\n== bounded exhaustive backward search =="
  traverse_ (\(n, budget) =>
      let (found, seenN, exhausted) = bfsRun n budget
      in putStrLn $ "  n=" ++ show n ++ ": "
           ++ (case found of
                 [] => "no product form"
                 fs => concat (intersperse ", "
                         (map (\((m, k), d) => show m ++ "*" ++ show k
                                            ++ " at distance " ++ show d) fs)))
           ++ "  (" ++ show seenN ++ " terms, "
           ++ (if exhausted then "space exhausted" else "budget cap") ++ ")")
    [(4, 30000), (5, 130000)]
```

Appendix C: The proof module, synchronized

The proof module Proofs.idr is a single artifact shared by every paper of this series. At this paper's first verification pass it stood at 940 lines and was reproduced here verbatim; it now stands at 3,337 lines across parts A to X, still %default total, still without postulates or holes, and still verified in full by the one command idris2 —check Proofs.idr. Reproducing it verbatim in every paper stopped scaling and stopped being honest, since a copy is stale the day after it is made; from this version on, the code of record is the module itself, which accompanies the series, and this appendix carries its map and this paper's anchors.

The module map, part banners with source lines: tier one, the shared calculus and its runs, lines 1 to 501; part A: R_arith as a relation, and fragment Theorem B, line 502; part B: the finite-exhaustion bridge, line 722; part C: split termination packaged as accessibility, line 760; part D: the divisibility probe, divmod correctness, line 778; part E: the linearity ladder (E8), line 937; part F: the soup absorber (E6), line 1041; part G: the circle and the octave (book chapters), line 1142; part H: the conserving chemistry balances (E6.2), line 1181; part I: the logic side (book chapters), line 1247; part J: the thrush (book chapter, the garden), line 1319; part K: agenda seeds (additive theorem B, cost of a box), line 1345; part L: no bird sings Peirce (L2), line 1403; part M: the workshop (book chapter two), line 1507; part N: the vanishing engine, verified (book chapter two), line 1541; part O: the coin pre-

served (L1, first half), line 1596; part P: the entropy budget (N4 weights, P1 reading), line 1659; part Q: the quoted alphabet (C5 stage one), line 1776; part R: the reader's engine (C5 stage two), line 1864; part S: every typed song ends (L1, second half), line 2051; part T: the first eye (C5 stage three, movement one), line 2230; part U: the second eye and the assembler (C5, movement two), line 2428; part V: two refusals, one ledger (occurrence conservation), line 2728; part W: the dispatcher (C5 stage three, movement three), line 2863; part X: the knot (C5 stage three, movement four), line 2983.

Anchors this paper relies on: tier one carries the calculus, the runs, and the numeral adequacy (num, numAdequate, skIdentity, iSteps, bSteps, fusionFact); part A carries the rewriting arithmetic as a relation with the fragment of Theorem B (the StepR relation, the inversion lemmas, boxReduces and the value ledger with afUnique); part B the finite-exhaustion bridge behind the backward searches; part C split termination as accessibility.

Developments since v0.4. Three later results touch this paper's subject and are reported in their own papers rather than duplicated here. The cost formula boxLen (module part K) prices the product box exactly, rLen of boxReduces m k equal to the successor of m times k plus two. The size formula tSizeNum (part P) gives every numeral its exact cell count, and the entropy budget built on it prices this paper's multiplication thermodynamically, with the finding that the fast operand order is the hot order. And the reader campaign (parts Q to X) closed with a verified self-reader for the calculus these numbers live in, so the terms defined here can now be read, quoted, and stepped by a term of their own language, with compiled correctness.

References

1. H. B. Curry and R. Feys, *Combinatory Logic*, Vol. I, North-Holland, 1958.
2. J. R. Hindley and J. P. Seldin, *Lambda-Calculus and Combinators: An Introduction*, Cambridge University Press, 2008.
3. R. P. Stanley, *Catalan Numbers*, Cambridge University Press, 2015.
4. P. Tarau, work on tree-represented natural numbers and their arithmetic, including hereditarily binary number systems (2013 onward).
5. H. P. Barendregt, *The Lambda Calculus: Its Syntax and Semantics*, revised edition, North-Holland, 1984.
6. F. Baader and T. Nipkow, *Term Rewriting and All That*, Cambridge University Press, 1998.
7. D. E. Knuth and P. B. Bendix, 'Simple word problems in universal algebras', in J. Leech (ed.), *Computational Problems in Abstract Algebra*, Pergamon Press, 1970, pages 263-297.
8. J.-L. Loday, 'Arithmetree', *Journal of Algebra* 258 (2002), 275-309.
9. A. Bruno and D. Yasaki, 'The arithmetic of trees', *Involve* 4 (2011), 1-11.
10. F. Luccio, 'Rooted tree arithmetic and equations', arXiv:1510.05512, 2015.
11. J. H. Conway, 'FRACTRAN: a simple universal programming language for arithmetic', in T. M. Cover and B. Gopinath (eds.), *Open Problems in Communication and Computation*, Springer, 1987, pages 4-26.
12. W. Ackermann, 'Die Widerspruchsfreiheit der allgemeinen Mengenlehre', *Mathematische Annalen* 114 (1937), 305-315.
13. M. Lothaire, *Combinatorics on Words*, Cambridge University Press, 1997.
14. K. Bimbó, *Combinatory Logic: Pure, Applied and Typed*, CRC Press, 2012.
15. Project secret2, 'Transparent Arithmetic over the Combinators K and S: Divisibility Automata, Factoring as Rewriting, and the Search for a Prime-Revealing Representation', working paper v0.8, August 11, 2026.
16. E. Brady, 'Idris 2: Quantitative Type Theory in Practice', *Proceedings of ECOOP 2021*, LIPIcs 194, 2021.